

How to use Python_STARS tool

Borja Barahona Gómez. Supervisors: Miguel Sanz Novo, Víctor M. Rivilla Rodríguez.

May 12, 2026

This tool for Python was developed to perform statistical analysis in the simulated spectrum of a template astronomical source. It combines the simulated spectra of all detected molecules and provides visualization and statistical analysis commands. Further details about the tool are presented below. **Previously, you need to run MADCUBA_STARS tool, as you need all the simulated spectrum of the molecules.**

1 Requirements.

You will need for this tool:

- Python 3.x.
- NumPy.
- Matplotlib.
- SciPy.
- MADCUBA_STARS simulated spectra files from a separate MADCUBA_STARS tool.

2 Steps for running the tool.

1. Download the package of the tool. This package must contain a file .py called *'python_stars.py'*. As a first step, you must run MADCUBA_STARS tool in order to obtain two types of files: a .txt file called 'molecular_gooddata.txt', and different files (more than 120) which are named 'simulate_generate_(name of molecule)'. All of these files you must localize them in a directory and move it to JupyterLab files, or see the path of the directory if you are using Pycharm (if so, you need to change the path of the lecture of files, it is indicated in the tool).
2. Change the path in which the files are located. In this example, I have put all my files in the JupyterLab directory. However, you can import all the files using other directories, only if you have defined them properly. The places of which you need to change the code are indicated as *(CHANGE)*: these are the lines 151-154.

```
1      molecula=[]
2      base_datos=[]
3      valor=[]
4      referencia=[]
5      inputText="moleculardata/molecular_gooddata.txt" # (CHANGE)
6      inputDirectory="moleculardata" # (CHANGE)
7      cloudName="G+0.673-0.027" # (CHANGE)
8      nlines=75600 # (CHANGE)
9      with open(inputText, "r", encoding="utf-8") as f:
10         (...)
```

3. You can run the code by calling the tool with a specified command (see all the different commands with the command 'help') and putting one or two inputs: if you want different options instead of default ones, and if so, put all the options you want. Otherwise, the tool will autocomplete them with default ones.

3 Commands of the tool: *help*.

You will find different statistical options here for the tool. If you want to put a command, just put this in another code cell:

```
1 program("%s" % command)
```

where the command could be all of the following subsections you will see before. Also, you can save the output of every command if you put the following command window:

```
1 program("%s",1 % command)
```

as there is a supplementary option for saving files and figures.

3.1 Command *plot_spectra*.

This command will plot the generated spectrum you are exporting. This will lead to the following inputs:

```
1 program("plot_spectra")
2 >>> Want some other parameters to fill the plot? If not, it will be filled by
   default values.
3 No
```

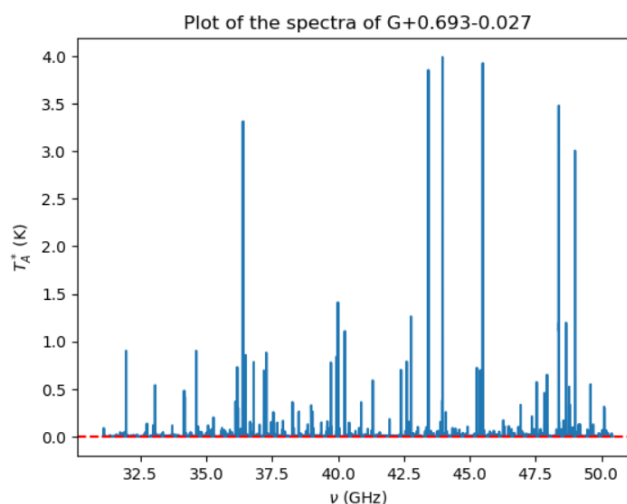


Figure 1: The plot of the simulated spectrum with default values.

You can use different options for this command:

- *w_units*: are the units of the frequency. You can put 'Hz', 'MHz' and 'GHz' (default). The data will be in this units.
- *grid*: it will put a grid in the graph. Default is 'No' or '0', but you can put 'Yes' or '1' and obtain the graph with a grid.
- *y_scale*: it will apply a scale for the Y axis. It is useful when you are trying to see molecules whose abundance is tiny. The default value for this option is 'linear', but you can put 'log' if you want a Y log scale.
- *wantzoom*: this option will zoom a part of the plot. If you put 1 or True, the program will request you as an input an array with the minimum value of both *x* and *y*, and also their maximum values with the syntax: [*x_{min}*, *x_{max}*, *y_{min}*, *y_{max}*]. Default value is 0 or False.

For instance, using all the commands mentioned before, we can obtain a plot of the spectrum as:

```
1 program("plot_spectra")
2 >>> Want some other parameters to fill the plot? If not, it will be filled by
   default values.
3 Yes
```

```

4      >>> Please put all the paramters here, separated by commas and without blanks
      . Check help if you want to fill all with your preferences.
5      The parameters for 'plot_spectra' are:
6      (w_units,grid,y_scale,wantzoom)
7      GHz,1,log,1
8      Where do you want the zooming? Put the values of xmin, xmax, ymin, ymax.
      Separate all the values by commas
9      35,35.2,1e-14,0.5
10     saved as: %show_spectra_g0693.svg%

```

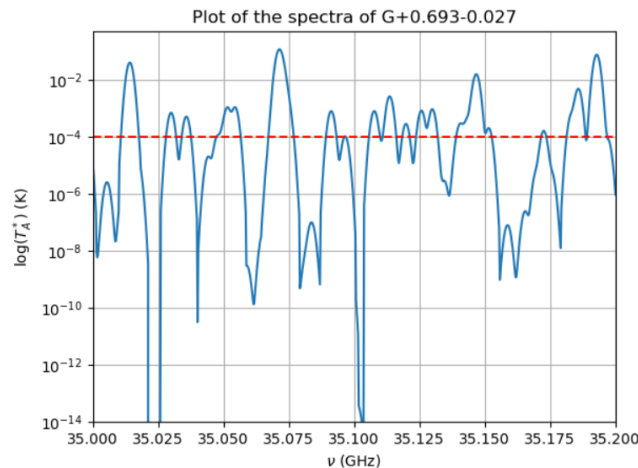


Figure 2: The plot of the simulated spectrum with different values. We depict with a red dashed line the threshold used for the simulated spectrum.

3.2 Command *frequency_lines*.

It will list all of the frequency lines of the whole spectrum. This command will get something like the following:

```

1      program("frequency_lines")
2      >>> Want some other parameters to fill the plot? If not, it will be filled by
      default values.
3      No

1      Obtained a peak in v= 31.111 GHz
2      (...)
3      Obtained a peak in v= 50.390 GHz

```

You can also use different options in this command:

- *h_min*: it is the minimum height of the peaks. The program will search for maximum values of intensity greater than this value. The greater the value of minimum heigh, the smaller the amount of peaks. Default value is 0.01.
- *threshold*: the threshold for the program of finding peaks. Default is 10^{-4} .
- *distance*: the ratio of distance between peaks the program will search for has to be greater than or equal to 1. Default value is 1.
- *prominence*: this value will search for the prominence of peaks: which is the difference between the surroundings of the peak and the peak. Default value is 0.
- *w_units*: are the units of the frequency. You can put '*Hz*', '*MHz*' and '*GHz*' (default). The data will be in the according units.

For instance, using the according parameters for the data of the testbed, we will lead to the following results:

```

1      program("frequency_lines")
2      >>> Want some other parameters to fill the plot? If not, it will be filled by
      default values.
3      Yes

```

```

4 >>> Please put all the parameters here, separated by commas and without blanks.
    Check help if you want to fill all with your preferences.
5 The parameters for 'frequency_lines' are:
6 (hmin,threshold,distance,prominence,w_units)
7 5e-3,0,1,0,GHz

```

```

1 Obtained a peak in v= 31.111 GHz
2 (...)
3 Obtained a peak in v= 50.424 GHz

```

3.3 Command `mol_detection`.

This command will plot the spectra adding dashed lines with the corresponding molecules associated with each transition. You can select all the molecules detected in MADCUBA_STARS tool, or just a few of them. Due to the resolution and the quality of the transitions, this command will generate a .svg file (vectorial image, in which you can zoom without losing resolution), and the output in JupyterLab will be unreadable because of the resolution acquired in the program. I suggest saving the file with the `outputSave` command activated, hence you need to put: `program("mol_detection",1)`

This command will get something like the following, obtaining figure 3:

```

1 program("mol_detection")
2 >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3 No

```

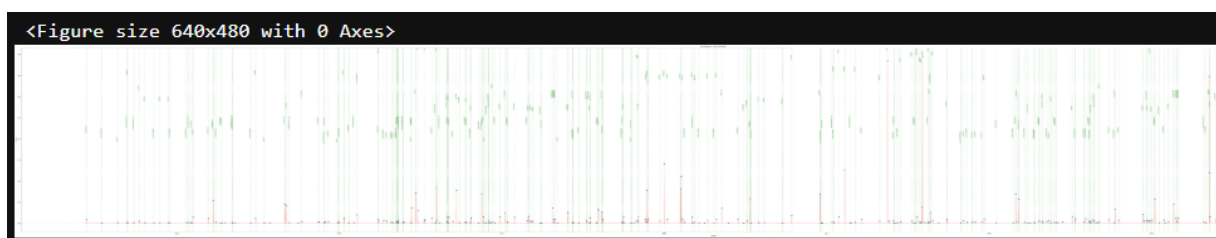
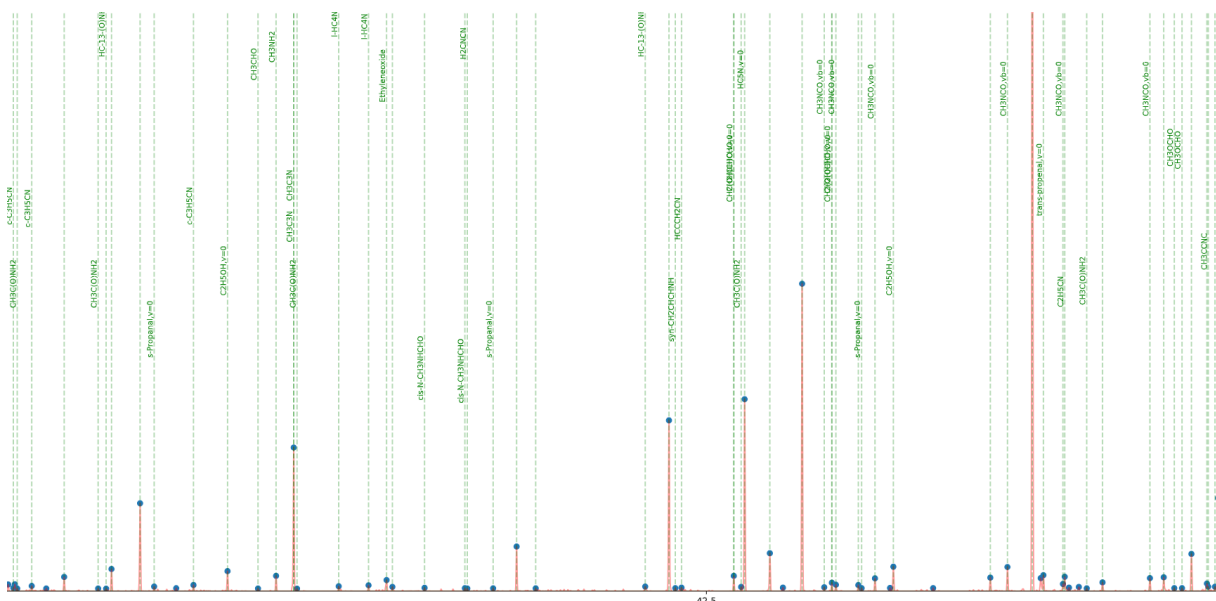


Figure 3: Output of the statistical analysis tool, which has high resolution and becomes unreadable. However, the tool creates a .svg file on which you can see properly the spectra with the corresponding molecules transitions.

But in the following figure, you can see the transitions labeled with the corresponding molecule:



This command has also different options, explained in the following list.

- *molecules_search*: the option for the search of molecules. Has two different inputs: 'All' (default), in which the spectrum will have labeled all the molecules transitions; and 'select', in which the command will ask you which molecules you want to search in the spectrum with an input. Then, it will generate the corresponding .svg file with the selected molecules.
- *error*: this option controls the tolerance of the intensity of peaks. If it is bigger, the necessary distance of the peaks will have to be greater than this tolerance, and you will obtain less transitions. Default value is 0.005.
- *h_min*: it is the minimum height of the peaks. The program will search for maximum values of intensity greater than this value. The greater the value of minimum height, the smaller the amount of peaks. Default value is 0.01.
- *threshold*: the threshold for the program of finding peaks. Default is 10^{-4} .
- *distance*: the ratio of distance between peaks the program will search for has to be greater than or equal to 1. Default value is 1.
- *prominence*: this value will search for the prominence of peaks: which is the difference between the surroundings of the peak and the peak. Default value is 0.
- *grid*: it will put a grid in the graph. Default is 'No' or '0', but you can put 'Yes' or '1' and obtain the graph with a grid.
- *y_scale*: it will apply a scale for the Y axis. It is useful when you are trying to see molecules whose abundance is tiny. The default value for this option is 'linear', but you can put 'log' if you want a Y log scale.
- *w_units*: are the units of the frequency. You can put 'Hz', 'MHz' and 'GHz' (default). The data will be in the according units.
- *wantzoom*: this option will zoom a part of the plot. If you put 1 or True, the program will request you as an input an array with the minimum value of both *x* and *y*, and also their maximum values with the syntax: $[x_{min}, x_{max}, y_{min}, y_{max}]$. Default value is 0 or False.

For instance, if you want to detect CH_3OCH_3 , $HNCO$, SiO , $HCOOH$, CH_3OH , C_2H_5CN , CH_2NH and HC_7N , we will have the following results:

```

1  program("mol_detection")
2  >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3  Yes
4  >>> Please put all the paramters here, separated by commas and without blanks.
    Check help if you want to fill all with your preferences.
5  The parameters for 'mol_detection' are:
6  (molecules_search,error,hmin,threshold,distance,prominence,grid,y_scale,w_units)
7  select,5e-3,1e-2,0,1,0,1,linear,GHz,1
8  Where do you want the zooming? Put the values of xmin, xmax, ymin, ymax. Separate
    all the values by commas
9  30,35,0,0.5
10 Which molecules you want to identify from the spectra? Separate them by commas
11 CH3OCH3,HNCO,SiO,HCOOH,CH3OH,C2H5CN,CH2NH,HC7N

```



Figure 5: Figure obtained with the command in JupyterLab. This will have a bad overview, but then in the .svg file saved you can see all the different transitions selected with the parameters.



Figure 6: Overview of the .svg file obtained by the program for detected molecules.

3.4 Command *density_lines*.

This command will find the density lines in different windows of the spectra. There will be in this tool two ways of defining windows: by "blocks", which will generate a linspace with a fixed number of equispaced values, or by "equispace", on which you can define a fixed separation between values. The way the density lines (d) are defined is:

$$d = \frac{\# \text{ of obtained peaks}}{\Delta\nu} \quad (1)$$

So, when you run this command, you will get a list of different windows with its associated density line d . This will be into two list-like outputs: frequencies of the edges of each window, and the density lines of each window.

```

1  program("density_lines")
2  >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3  No

1  ([31.57467984073538,
2  (...),
3  49.574679840735385],
4  [8.999999999999968,
5  (...),
6  19.0])

```

This command will have different options, explained in the following list:

- *h_min*: it is the minimum height of the peaks. The program will search for maximum values of intensity greater than this value. The greater the value of minimum height, the smaller the amount of peaks. Default value is 0.01.
- *threshold*: the threshold for the program of finding peaks. Default is 10^{-4} .
- *distance*: the ratio of distance between peaks the program will search for has to be greater than or equal to 1. Default value is 1.
- *prominence*: this value will search for the prominence of peaks: which is the difference between the surroundings of the peak and the peak. Default value is 0.
- *w_units*: are the units of the frequency. You can put 'Hz', 'MHz' and 'GHz' (default). The data will be in the according units.
- *search_method*: it will create different windows. The only two options for this command are "blocks" (it will create a linspace of equispaced values of ν) and "equispace" (it will create an arange of ν values spaced by the value you want to fix). Default value is blocks.
- $\Delta\nu$: if *search_method*=blocks, it will be the number of windows of the spectrum you want to crop. If it is *search_method*=equispace, it will be the separation of the equispaced values of ν . For blocks, default value is 20, for equispace default value is 0.5 GHz

Putting in practice all of these options, we have the next example:

```

1  Want some other parameters for this tool? If not, it will be filled by default
    values.
2  Yes
3  Please put all the parameters here, separated by commas and without blanks. Check
    help if you want to fill all with your preferences.
4  The parameters for 'density_lines' are:
5  (hmin,threshold,distance,prominence,w_units,search_method,distance of arange/
    number of blocks)
6  1e-2,1e-4,1,0,GHz,blocks,20

1  ([32.09313034139378,
2  (...),
3  49.40678885258653],
4  [11.29166316140607,
5  (...),
6  21.110500693063486])

```

3.5 Command *histogram*.

It will generate a histogram with the density lines of each window. It will request the method for creating the windows seen in Sec. 3.4: "blocks" or "equispace". For instance:

```

1  program("histogram")
2  >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3  No

```

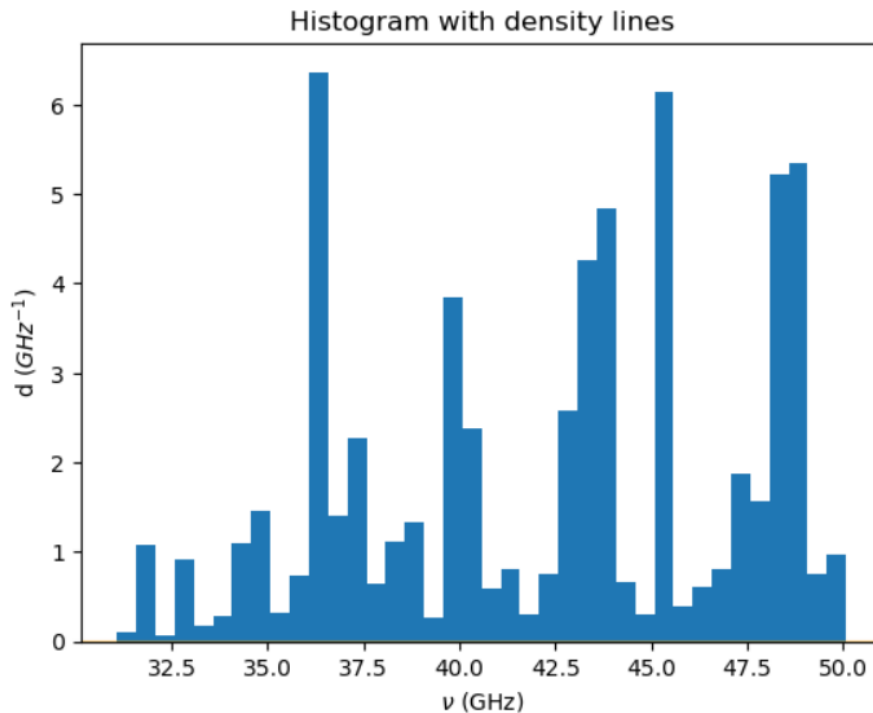


Figure 7: Overview of the histogram of density lines for each window obtained by the program.

The different options for histogram with its default values are:

- *h_min*: it is the minimum height of the peaks. The program will search for maximum values of intensity greater than this value. The greater the value of minimum height, the smaller the amount of peaks. Default value is 0.01.
- *threshold*: the threshold for the program of finding peaks. Default is 10^{-4} .

- *distance*: the ratio of distance between peaks the program will search for has to be greater than or equal to 1. Default value is 1.
- *prominence*: this value will search for the prominence of peaks: which is the difference between the surroundings of the peak and the peak. Default value is 0.
- *w_units*: are the units of the frequency. You can put 'Hz', 'MHz' and 'GHz' (default). The data will be in the according units.
- *search_method*: it will create different windows. The only two options for this command are "blocks" (it will create a linspace of equispaced values of ν) and "equispace" (it will create an arange of ν values spaced by the value you want to fix). Default value is blocks.
- $\Delta\nu$: if *search_method*=*blocks*, it will be the number of windows of the spectrum you want to crop. If it is *search_method*=*equispace*, it will be the separation of the equispaced values of ν . For blocks, default value is 20, for equispace default value is 0.5 GHz

Then, you can put as an example the following input:

```

1  program("histogram")
2  >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3  No
4  Please put all the parameters here, separated by commas and without blanks.
    Check help if you want to fill all with your preferences.
5  The parameters for 'histogram' are:
6  (hmin,threshold,distance,prominence,w_units,search_method,distance of arange/
    number of blocks)
7  1e-2,1e-4,1,0,GHz,equispace,0.2

```

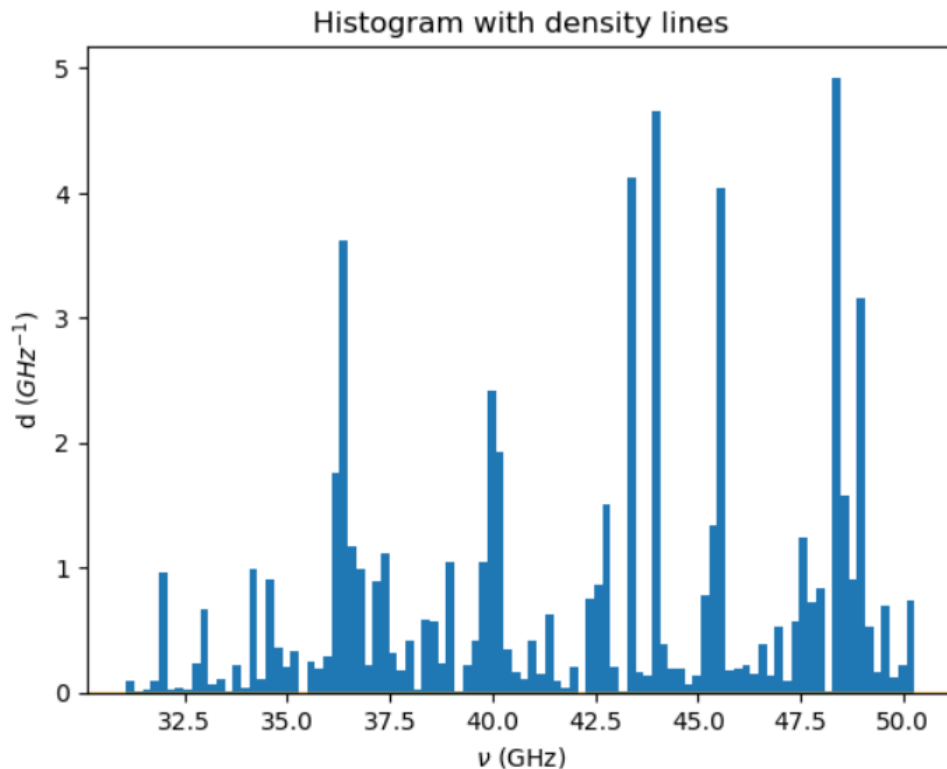


Figure 8: Overview of the histogram of density lines for each window obtained by the program, with the fixed options before.

There is a nice check, which is putting histograms of different window width. In fact, you can see that the density lines are approximately bounded by less than 6, and matches with the upper bound of the histogram with a default value of window width.

3.6 Command `subplots_low_lines`.

This command will generate plenty of subplots of each window which has low density of lines. This will be the generalized version of the next command `zoom_low_lines`.

The default version of this command is:

```

1  program("subplots_low_lines")
2  >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3  No

```

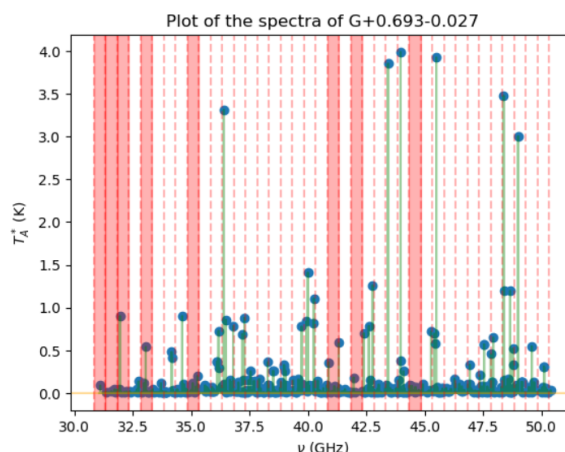


Figure 9: Overview of the spectra selecting the windows with low density of lines.

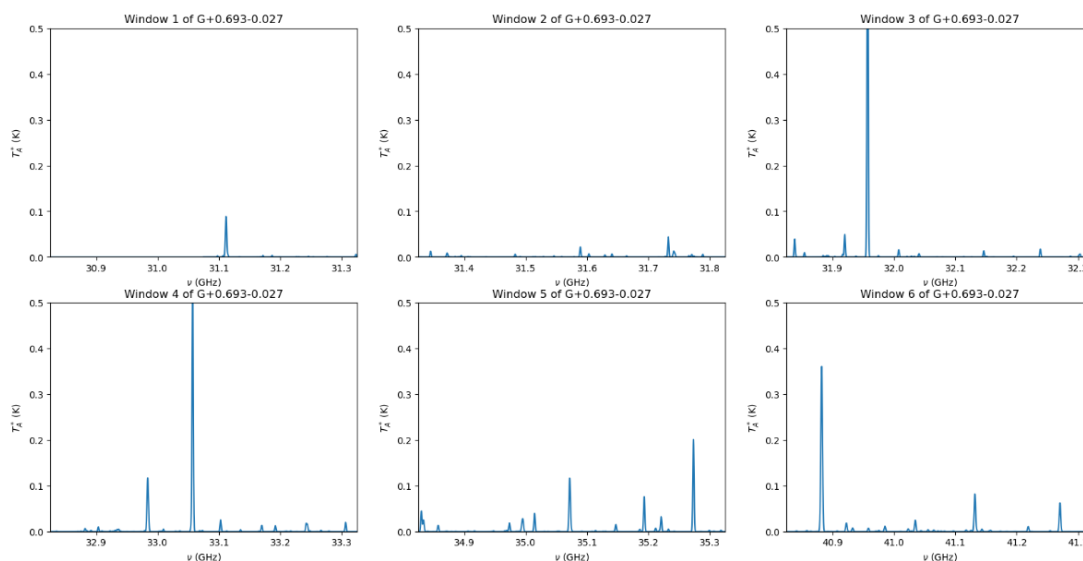


Figure 10: Overview of the subplots of some of the windows with low density of lines.

You can select different options, for this and the rest of command related with the selection of density lines: `zoom_low_lines` and `zoom_high_lines`:

- `grid`: it will put a grid in the graph. Default is 'No' or '0', but you can put 'Yes' or '1'.
- `y_scale`: it will apply a scale for the Y axis. It is useful when you are trying to see molecules whose abundance is tiny. The default value for this option is 'linear', but you can put 'log' if you want a Y log scale.
- `selection_pass`: the percentage of windows you want to pass as low density lines zones. If it is higher, there will be more windows classified as low density lines windows. Default value is 20 %.

- *h_min*: it is the minimum height of the peaks. The program will search for maximum values of intensity greater than this value. The greater the value of minimum height, the smaller the amount of peaks. Default value is 0.01.
- *threshold*: the threshold for the program of finding peaks. Default is 10^{-4} .
- *distance*: the ratio of distance between peaks the program will search for has to be greater than or equal to 1. Default value is 1.
- *prominence*: this value will search for the prominence of peaks: which is the difference between the surroundings of the peak and the peak. Default value is 0.
- *w_units*: are the units of the frequency. You can put 'Hz', 'MHz' and 'GHz' (default). The data will be in the according units.
- *selection_method*: the method of which you classify the low density lines windows. Most accurate method is "weighted", which takes a probability distribution as:

$$P_{\alpha} = \sum_{p_i \in \text{window}} I_i^{-\alpha} \cdot \#_i, \quad (2)$$

where $\#_i$ is the number of lines with intensity I_i and α is the parameter of the probability distribution on which you can control the weight of the peaks. If $\alpha > 0$, greater peaks will count on the classification less than lower peaks. However, if $\alpha < 0$, greater peaks will be taken into account more than smaller peaks.

If you want to classify the windows by other methods, you have "max_I", which takes the intensity average of each window and takes the classification by this discrimination parameter. Nevertheless, you can have the case $\alpha = 0$ (default method), which is called as *max_peaks*: it classifies each window by the number of peaks obtained.

- *search_method*: it will create different windows. The only two options for this command are "blocks" (it will create a linspace of equispaced values of ν) and "equispace" (it will create an arange of ν values spaced by the value you want to fix). Default value is blocks.
- $\Delta\nu$: if *search_method*=*blocks*, it will be the number of windows of the spectrum you want to crop. If it is *search_method*=*equispace*, it will be the separation of the equispaced values of ν . For blocks, default value is 20, for equispace default value is 0.5 GHz

For instance, we can use this options for this command, and obtain:

```

1  program("subplots_low_lines")
2  >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3  Yes
4      Please put all the parameters here, separated by commas and without blanks.
    Check help if you want to fill all with your preferences.
5      The parameters for 'subplots_low_lines' are:
6      (grid,y_scale,intensityaverage_pass,hmin,threshold,distance,prominence,w_units,
    search_method,distance of arange/number of blocks)
7      1,log,5,1e-2,3e-4,1,0,GHz,max_I,equispace,0.15

```

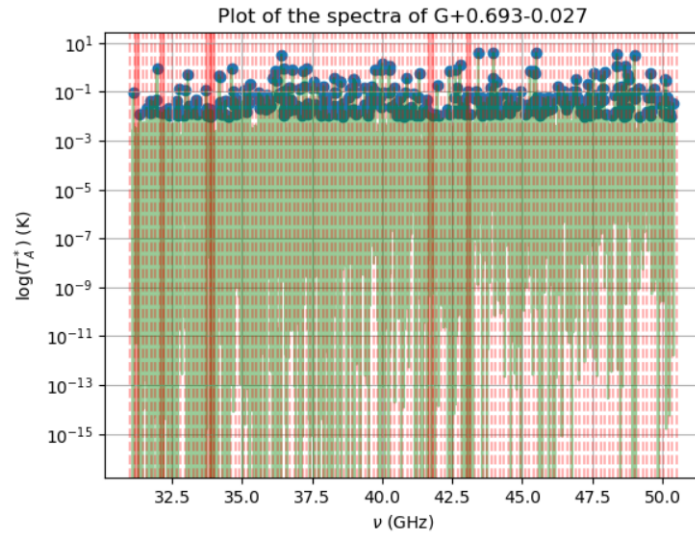


Figure 11: Overview of the spectra selecting the windows with low density of lines, using different commands.

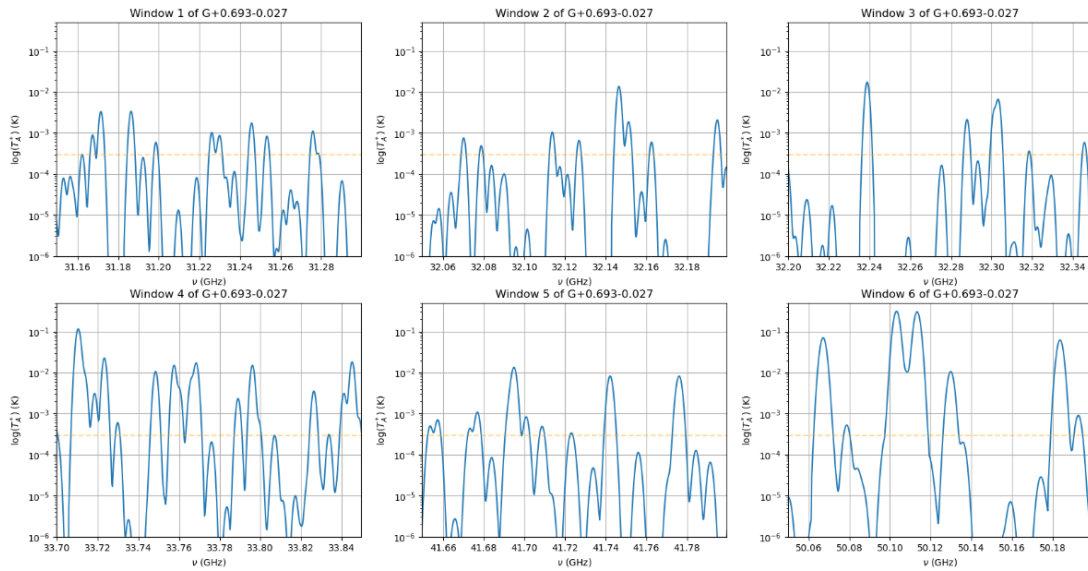


Figure 12: Overview of the subplots of some of the windows with low density of lines, using different commands. We depict with an orange dashed line the threshold used for the simulated spectrum.

3.7 Command `zoom_low_lines`.

For this and the next option, you will have the same options as `subplots_low_lines`. However, now you can select the window for an individual plot, selecting which window do you want to plot. For default values:

```

1  program("zoom_low_lines")
2  >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3  No
4  There are 13 windows
5  Do you want to choose any window? Choose 'Yes' or 'No' for this question.
6  No
7  Then you are done!

```

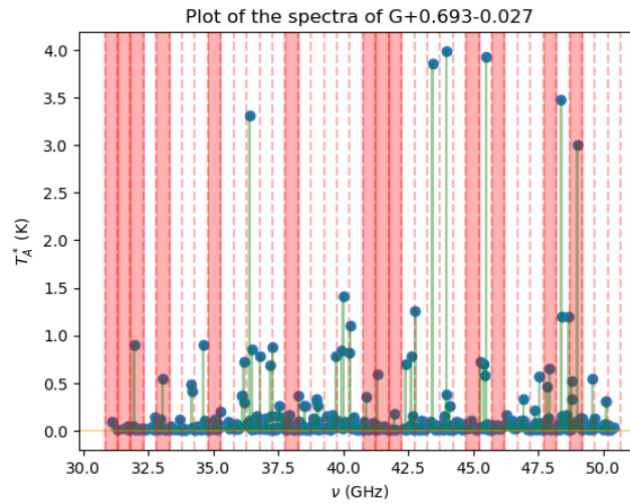


Figure 13: Overview of the spectra selecting the windows with low density of lines.

And for selected options, where you can find them at Sec. 3.6, we have:

```

1  program("zoom_low_lines")
2  >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3  Yes
4  Please put all the parameters here, separated by commas and without blanks. Check
    help if you want to fill all with your preferences.
5  The parameters for 'zoom_low_lines' are:
6  (grid,y_scale,intensityaverage_pass,hmin,threshold,distance,prominence,w_units,
    search_method,distance of arange/number of blocks)
7  1,linear,5,1e-2,0,1,0,GHz,max_I,equispace,0.5
8  There are 13 windows
9  Do you want to choose any window? Choose 'Yes' or 'No' for this question.
10 Yes
11 Which window do you want to choose? From 1 to 13
12 10

```

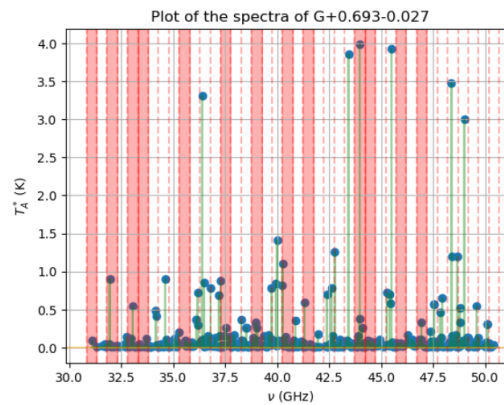


Figure 14: Overview of the spectra selecting the windows with low density of lines, with all the commands selected before.

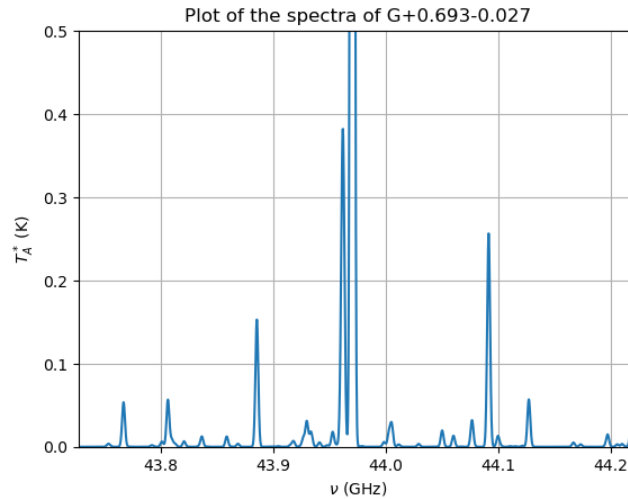


Figure 15: Window selected in the input of this command.

3.8 Command *zoom_high_lines*.

This command has exactly the same options and inputs as Sec. 3.7. Then, go for the documentation of this command to this other related command.

3.9 Command *gaussian_kde_map*.

This command performs a representation of a kernel-density estimate using gaussian kernels. Kernel density estimation is a way to estimate the probability density function (PDF) of a random variable in a non-parametric way. For using this command, you will have to put in a command window:

```

1  program("gaussian_kde_map")
2  >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3  No

```

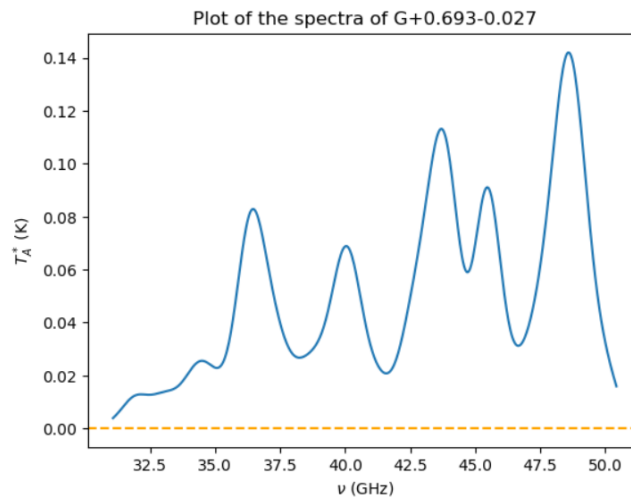


Figure 16: Figure of the probability density function of the spectra, using a bandwidth of 0.1. We depict with an orange dashed line the threshold used for the simulated spectrum.

The different options for this command are:

- *w_units*: are the units of the frequency. You can put 'Hz', 'MHz' and 'GHz' (default). The data will be in the according units.
- *grid*: it will put a grid in the graph. Default is 'No' or '0', but you can put 'Yes' or '1' and obtain the graph with a grid.

- *y_scale*: it will apply a scale for the Y axis. It is useful when you are trying to see molecules whose abundance is tiny. The default value for this option is 'linear', but you can put 'log' if you want a Y log scale.
- *band_width*: critical parameter for the smooth of the density. If higher, you will get a higher smooth spectra, but if lower, you will see more and more peaks. Default value is 0.1.

Using other options, you will get:

```

1  program("gaussian_kde_map")
2  >>> Want some other parameters to fill the plot? If not, it will be filled by
    default values.
3  Yes
4  Please put all the parameters here, separated by commas and without blanks. Check
    help if you want to fill all with your preferences.
5  The parameters for 'gaussian_kde_map' are:
6  (w_units,grid,y_scale,gaussian_method)
7  GHz,1,linear,0.03

```

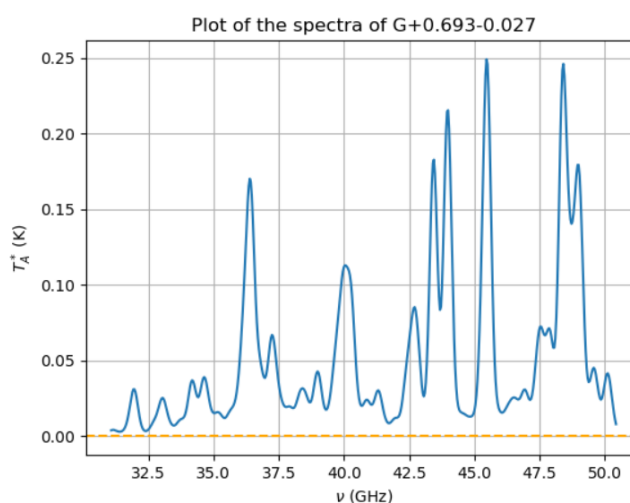


Figure 17: Figure of the probability density function of the spectra, using a bandwidth of 0.03. We depict with an orange dashed line the threshold used for the simulated spectrum.

If you find any issues with the tool, please write to the mail bbarahon@ucm.es. Alternatively, you may reach out to the CAB supervisors at msanz@cab.inta-csic.es or vrivilla@cab.inta-csic.es.

References

1. S. Martín, J. Martín-Pintado, C. Blanco-Sánchez, V. M. Rivilla, A. Rodríguez-Franco, and F. Rico-Villas. Spectral Line Identification and Modelling (SLIM) in the MAdrid Data CUBe Analysis (MADCUBA) package: Interactive software for data cube analysis. 631:A159, 2019. ISSN 0004-6361, 1432-0746. doi: 10.1051/0004-6361/201936144. URL <https://www.aanda.org/10.1051/0004-6361/201936144>.

NO INCLUIR EN EL DOC FINAL, ODIO BIBTEX Y HACE COSAS MALAS

References